```
"""
Telegram Bot — Claude AI Chat + Crypto Trade Alerts
====================================================
Features:
  - Chat with Claude AI directly from Telegram
  - Automatic crypto price scanning every 5 minutes
  - Trade alerts sent to your Telegram when conditions are met
  - Commands: /price, /scan, /status, /help

Install:
  pip install pytelegrambotapi anthropic requests schedule

Environment variables to set in Railway:
  TELEGRAM_TOKEN   = your bot token
  TELEGRAM_CHAT_ID = your personal chat ID
  ANTHROPIC_API_KEY = your Claude API key
"""

import os
import time
import threading
import logging
import requests
import schedule
from datetime import datetime

import telebot
import anthropic

# ── Logging ──────────────────────────────────────────────────────────────
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s  %(levelname)-8s  %(message)s",
    handlers=[logging.StreamHandler(), logging.FileHandler("bot.log")],
)
log = logging.getLogger(__name__)

# ═══════════════════════════════════════════════════════════════════════
#  CONFIG  — set these as environment variables in Railway
# ═══════════════════════════════════════════════════════════════════════
TELEGRAM_TOKEN   = os.getenv("TELEGRAM_TOKEN", "8760960206:AAHFpgLemXMEz_jhwa6clS
TELEGRAM_CHAT_ID = os.getenv("TELEGRAM_CHAT_ID", "8651075661")
ANTHROPIC_API_KEY = os.getenv("ANTHROPIC_API_KEY", "")
```

```
# Coins to watch
WATCHLIST = ["BTC", "ETH", "SOL", "LINK"]

# Alert thresholds
RSI_BUY_THRESHOLD  = 40   # Alert when RSI drops below this (oversold)
RSI_SELL_THRESHOLD = 65   # Alert when RSI rises above this (overbought)
SCAN_INTERVAL_MIN  = 5    # How often to scan (minutes)

# ═══════════════════════════════════════════════════════════════════
#  SETUP
# ═══════════════════════════════════════════════════════════════════
bot    = telebot.TeleBot(TELEGRAM_TOKEN)
client = anthropic.Anthropic(api_key=ANTHROPIC_API_KEY)

# Store conversation history per user for Claude context
conversation_history = {}


# ═══════════════════════════════════════════════════════════════════
#  MARKET DATA  (free CoinGecko API — no key needed)
# ═══════════════════════════════════════════════════════════════════
COIN_IDS = {
    "BTC": "bitcoin",
    "ETH": "ethereum",
    "SOL": "solana",
    "LINK": "chainlink",
}

def get_price(symbol: str) -> dict:
    """Fetch current price and 24h change from CoinGecko."""
    coin_id = COIN_IDS.get(symbol.upper())
    if not coin_id:
        return {}
    try:
        url = f"https://api.coingecko.com/api/v3/simple/price"
        params = {
            "ids": coin_id,
            "vs_currencies": "usd",
            "include_24hr_change": "true",
            "include_24hr_vol": "true",
        }
        r = requests.get(url, params=params, timeout=10)
        data = r.json().get(coin_id, {})
        return {
            "symbol":  symbol.upper(),
            "price":   data.get("usd", 0),
            "change":  data.get("usd_24h_change", 0),
```

```
            "volume":  data.get("usd_24h_vol", 0),
        }
    except Exception as e:
        log.error(f"Price fetch error for {symbol}: {e}")
        return {}


def get_rsi(symbol: str, period: int = 14) -> float:
    """Calculate RSI using daily closes from CoinGecko."""
    coin_id = COIN_IDS.get(symbol.upper())
    if not coin_id:
        return 50.0
    try:
        url = f"https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart"
        params = {"vs_currency": "usd", "days": 30, "interval": "daily"}
        r = requests.get(url, params=params, timeout=10)
        prices = [p[1] for p in r.json().get("prices", [])]
        if len(prices) < period + 1:
            return 50.0

        deltas = [prices[i] - prices[i-1] for i in range(1, len(prices))]
        gains  = [max(d, 0) for d in deltas]
        losses = [abs(min(d, 0)) for d in deltas]

        avg_gain = sum(gains[-period:]) / period
        avg_loss = sum(losses[-period:]) / period

        if avg_loss == 0:
            return 100.0
        rs  = avg_gain / avg_loss
        rsi = 100 - (100 / (1 + rs))
        return round(rsi, 1)
    except Exception as e:
        log.error(f"RSI error for {symbol}: {e}")
        return 50.0


# ═══════════════════════════════════════════════════════════════════
#  CLAUDE AI
# ═══════════════════════════════════════════════════════════════════
def ask_claude(user_id: str, message: str) -> str:
    """Send a message to Claude, maintaining conversation history."""
    if user_id not in conversation_history:
        conversation_history[user_id] = []

    conversation_history[user_id].append({
        "role": "user",
```

```
        "content": message
    })

    # Keep last 20 messages to avoid token limits
    history = conversation_history[user_id][-20:]

    try:
        response = client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=1024,
            system=(
                "You are a helpful trading assistant and general AI. "
                "You help with crypto/stock questions, market analysis, and gener
                "Keep responses concise and clear — this is a Telegram chat so av
                "Use simple formatting that works in Telegram (no markdown header
            ),
            messages=history,
        )
        reply = response.content[0].text

        conversation_history[user_id].append({
            "role": "assistant",
            "content": reply
        })

        return reply
    except Exception as e:
        log.error(f"Claude error: {e}")
        return "Sorry, I couldn't reach Claude right now. Try again in a moment."


def claude_market_analysis(symbol: str, price: float, change: float, rsi: float)
    """Ask Claude to analyze a specific coin's current conditions."""
    prompt = (
        f"Quick analysis of {symbol}:\n"
        f"Price: ${price:,.2f}\n"
        f"24h change: {change:.1f}%\n"
        f"RSI: {rsi}\n\n"
        f"In 2-3 sentences, what does this suggest? Should I be cautious, watchin
    )
    try:
        response = client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=200,
            messages=[{"role": "user", "content": prompt}],
        )
        return response.content[0].text
```

```
    except:
        return "Analysis unavailable."


# ═══════════════════════════════════════════════════════════════
#  ALERT SCANNER
# ═══════════════════════════════════════════════════════════════
def scan_and_alert():
    """Scan watchlist and send alerts if conditions are met."""
    log.info("Running market scan...")
    for symbol in WATCHLIST:
        try:
            data = get_price(symbol)
            if not data:
                continue
            rsi = get_rsi(symbol)

            price  = data["price"]
            change = data["change"]

            # BUY alert condition
            if rsi <= RSI_BUY_THRESHOLD:
                analysis = claude_market_analysis(symbol, price, change, rsi)
                msg = (
                    f"🟢 BUY ALERT — {symbol}\n\n"
                    f"💰 Price: ${price:,.2f}\n"
                    f"📉 24h Change: {change:.1f}%\n"
                    f"📊 RSI: {rsi} (oversold)\n\n"
                    f"🤖 Claude says:\n{analysis}\n\n"
                    f"⚠️ This is not financial advice. Your decision."
                )
                bot.send_message(TELEGRAM_CHAT_ID, msg)
                log.info(f"BUY alert sent for {symbol}")

            # SELL alert condition
            elif rsi >= RSI_SELL_THRESHOLD:
                analysis = claude_market_analysis(symbol, price, change, rsi)
                msg = (
                    f"🔴 SELL ALERT — {symbol}\n\n"
                    f"💰 Price: ${price:,.2f}\n"
                    f"📈 24h Change: {change:.1f}%\n"
                    f"📊 RSI: {rsi} (overbought)\n\n"
                    f"🤖 Claude says:\n{analysis}\n\n"
                    f"⚠️ This is not financial advice. Your decision."
                )
                bot.send_message(TELEGRAM_CHAT_ID, msg)
                log.info(f"SELL alert sent for {symbol}")
```

```
            except Exception as e:
                log.error(f"Scan error for {symbol}: {e}")

    log.info("Scan complete.")


def run_scheduler():
    """Run the scanner on a schedule in a background thread."""
    schedule.every(SCAN_INTERVAL_MIN).minutes.do(scan_and_alert)
    # Run once immediately on startup
    scan_and_alert()
    while True:
        schedule.run_pending()
        time.sleep(30)


# ═══════════════════════════════════════════════════════════════════
#  TELEGRAM COMMANDS
# ═══════════════════════════════════════════════════════════════════
@bot.message_handler(commands=["start", "help"])
def handle_help(message):
    text = (
        "👋 Hey! I'm your trading assistant bot.\n\n"
        "Here's what I can do:\n\n"
        "📊 /price BTC — Get current price + RSI\n"
        "🔍 /scan — Manually run a market scan now\n"
        "📈 /status — See all watchlist coins\n"
        "🤖 Just type anything — Chat with Claude AI\n\n"
        f"🔔 Auto-scanning every {SCAN_INTERVAL_MIN} min\n"
        f"Watching: {', '.join(WATCHLIST)}"
    )
    bot.reply_to(message, text)


@bot.message_handler(commands=["price"])
def handle_price(message):
    parts = message.text.strip().split()
    symbol = parts[1].upper() if len(parts) > 1 else "BTC"

    bot.reply_to(message, f"Fetching {symbol} data...")

    data = get_price(symbol)
    if not data:
        bot.reply_to(message, f"Couldn't find price for {symbol}. Try BTC, ETH, o
        return
```

```
    rsi    = get_rsi(symbol)
    price  = data["price"]
    change = data["change"]
    emoji  = "📈" if change > 0 else "📉"

    condition = "Neutral"
    if rsi <= RSI_BUY_THRESHOLD:
        condition = "🟢 Oversold — possible buy zone"
    elif rsi >= RSI_SELL_THRESHOLD:
        condition = "🔴 Overbought — possible sell zone"

    text = (
        f"{emoji} {symbol}\n\n"
        f"💰 ${price:,.2f}\n"
        f"24h: {change:+.1f}%\n"
        f"RSI: {rsi}\n"
        f"Signal: {condition}"
    )
    bot.reply_to(message, text)


@bot.message_handler(commands=["scan"])
def handle_scan(message):
    bot.reply_to(message, "🔍 Running scan now...")
    scan_and_alert()
    bot.reply_to(message, "✅ Scan complete. Alerts sent if conditions were met.")


@bot.message_handler(commands=["status"])
def handle_status(message):
    bot.reply_to(message, "Fetching all coins...")
    lines = []
    for symbol in WATCHLIST:
        data = get_price(symbol)
        if not data:
            continue
        rsi    = get_rsi(symbol)
        price  = data["price"]
        change = data["change"]
        emoji  = "📈" if change > 0 else "📉"
        lines.append(f"{emoji} {symbol}: ${price:,.2f} ({change:+.1f}%) RSI:{rsi}")

    text = "📊 Market Status\n\n" + "\n".join(lines)
    bot.reply_to(message, text)


@bot.message_handler(func=lambda m: True)
```

```
def handle_message(message):
    """Any non-command message goes to Claude."""
    user_id = str(message.from_user.id)
    user_text = message.text

    # Show typing indicator
    bot.send_chat_action(message.chat.id, "typing")

    reply = ask_claude(user_id, user_text)
    bot.reply_to(message, reply)


# ═══════════════════════════════════════════════════════════════
#  MAIN
# ═══════════════════════════════════════════════════════════════
if __name__ == "__main__":
    log.info("Bot starting...")

    # Send startup message
    try:
        bot.send_message(
            TELEGRAM_CHAT_ID,
            f"🚀 Bot is online!\n"
            f"Watching: {', '.join(WATCHLIST)}\n"
            f"Scanning every {SCAN_INTERVAL_MIN} minutes\n"
            f"Type /help to see commands."
        )
    except Exception as e:
        log.error(f"Startup message failed: {e}")

    # Start scanner in background thread
    scanner_thread = threading.Thread(target=run_scheduler, daemon=True)
    scanner_thread.start()

    # Start Telegram bot (blocking)
    log.info("Listening for messages...")
    bot.infinity_polling()
```